

Algorithmique et Programmation orientée-objet

Ch.7 – Structures de données et Java Collections Framework

Bruno Quoitin
(bruno.quoitin@umons.ac.be)

Table des Matières

- **Introduction**
- **Collections**
 - Listes
 - Ensembles
- **Associations**

Introduction

- **Objectifs**

- L'objectif de ce chapitre est d'introduire le **Java Collections Framework** (JCF), un groupe d'interfaces, de classes et d'algorithmes de la bibliothèque Java permettant de manipuler facilement et efficacement des ensembles de données.
- Ce chapitre permet de comprendre le fonctionnement et l'utilisation de certaines des structures de données mises à disposition par le JCF telles que les listes chaînées, les tableaux dynamiques et les tables de hachage.

Introduction

- **Collections**

- Une **collection** est un terme générique désignant un objet regroupant de multiples éléments en un seul. Cet objet permet
 - le stockage,
 - l'interrogation,
 - la manipulation
 - et le parcours de ces éléments.
- Les collections sont subdivisées en plusieurs types abstraits de données tels que les **listes**, les **ensembles** et les **files**, offrant chacun des services spécifiques.

Table des Matières

- **Introduction**
- **Collections**
 - Listes
 - Ensembles
 - Files
- **Associations**

Java Collections Framework

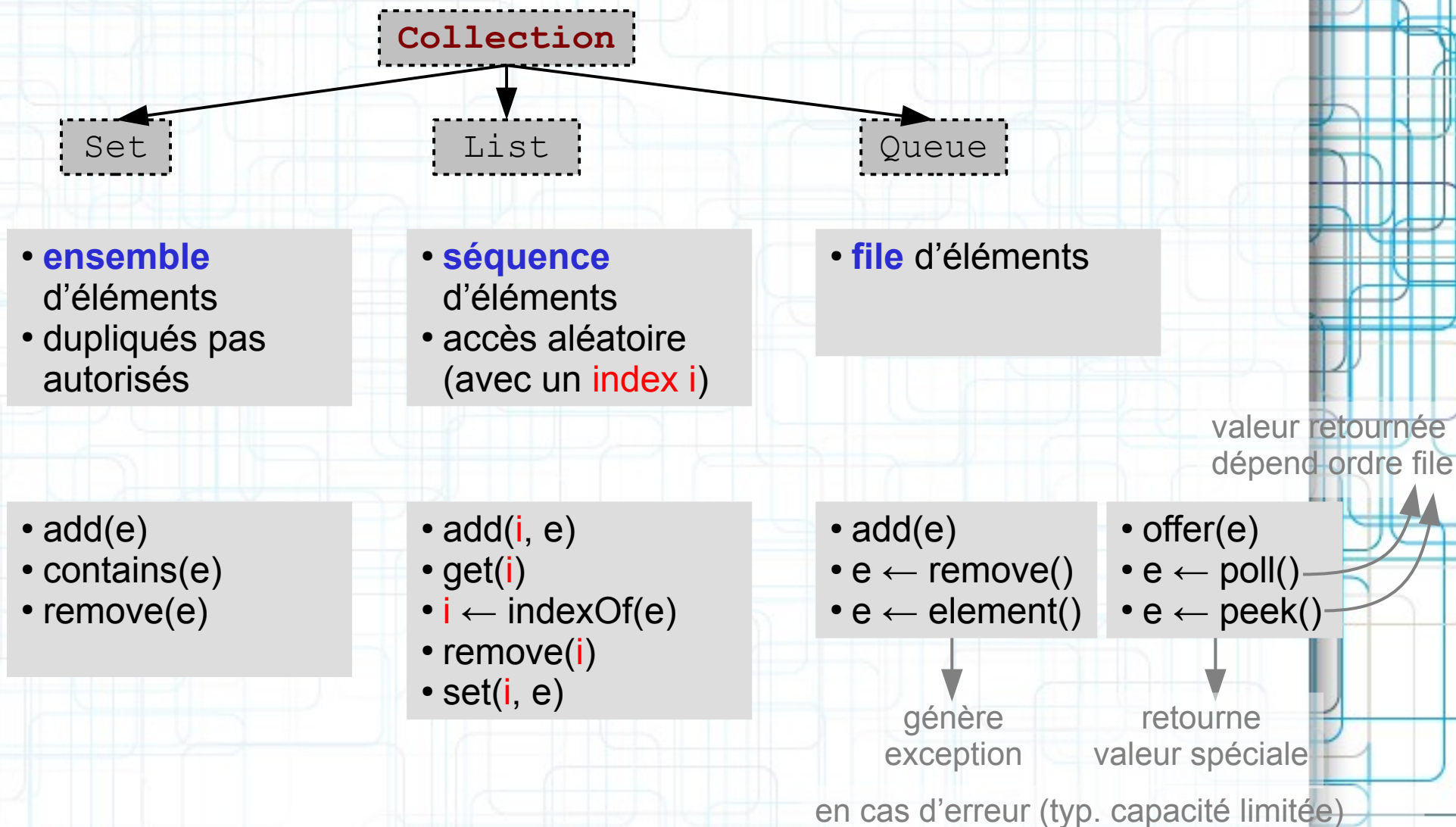
- **Interface Collection**

- Le plus petit élément commun aux collections du JCF est l'interface **Collection** (`java.util`). Cette interface définit les opérations liées à la gestion d'un ensemble quelconque d'éléments.

```
public interface Collection<E>
    extends Iterable<E> {
        boolean    add(E o);
        boolean    addAll(Collection<? extends E> c);
        void       clear();
        boolean    contains(E o);
        boolean    isEmpty();
        Iterator<E> iterator();
        boolean    remove(E o);
        boolean    removeAll(Collection<?> c);
        int        size();
        Stream<E>  stream();
        Object[]   toArray();
        <T> T[]     toArray(T[] a);
        /* ... */
    }
```

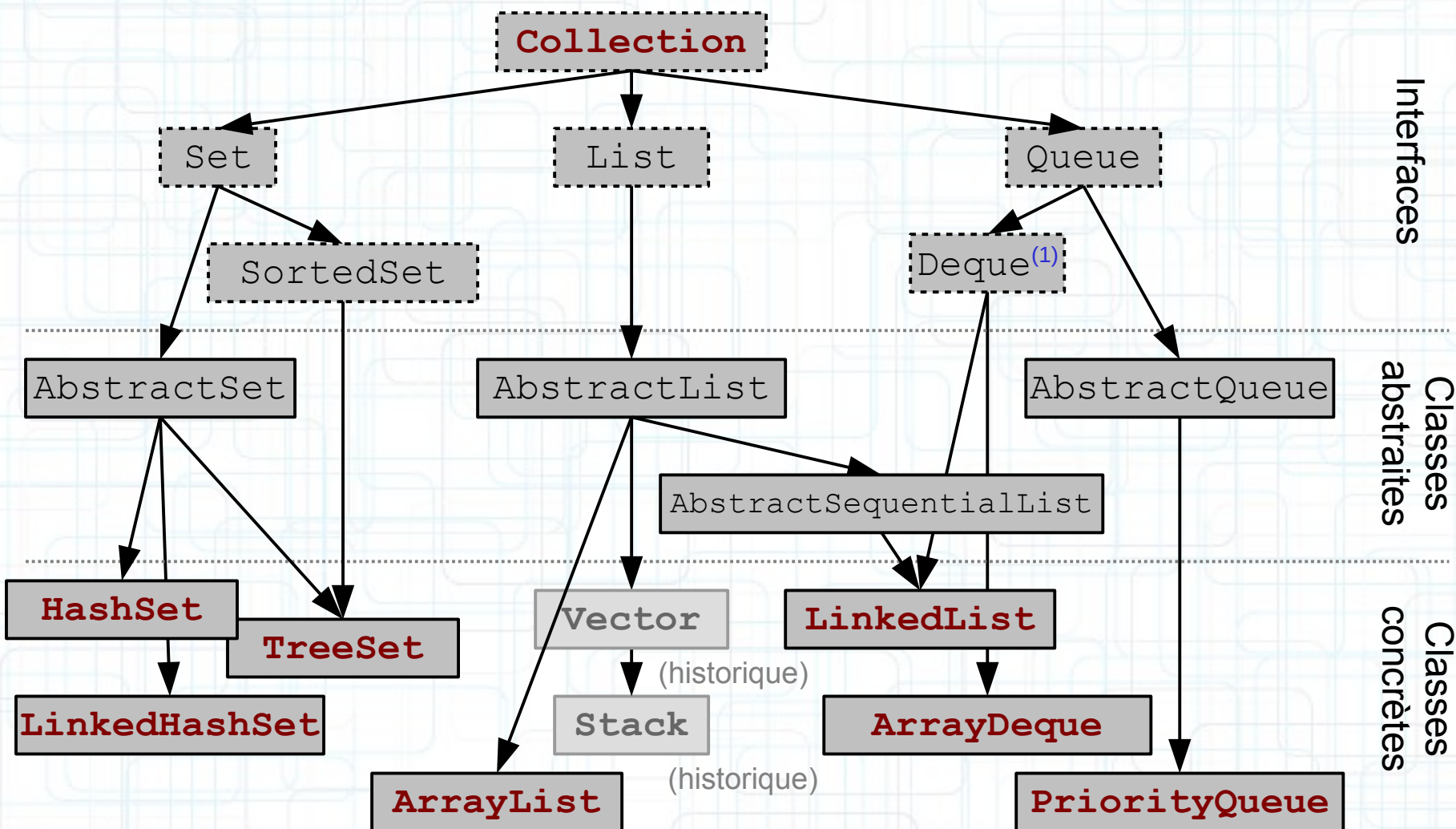
Java Collections Framework

• Hiérarchie de classes Collection



Java Collections Framework

- Hiérarchie de classes Collection



(1) DeQue = Double-ended Queue, « deck »

Java Collections Framework

- Hiérarchie de classes Collection

		dupli qués	null	ordon né	synch ronisé	remarques
List	ArrayList	✓	✓			tableau dynamique
	Vector	✓	✓		✓	tableau dynamique, (plus de contrôle que ArrayList)
	Stack	✓	✓		✓	LIFO (basé sur Vector)
Deque	LinkedList	✓	✓			liste doublement chaînée
	ArrayDeque	✓				FIFO, LIFO, ... (basé sur un tableau)
	PriorityQueue	✓				tas binaire
Queue	HashSet					table de hachage
	TreeSet			✓ (1)		arbre binaire (rouge-noir)
	LinkedHashSet			✓ (2)		table de hachage + liste doublement chaînée
Set						

(1) ordre naturel ; (2) ordre d'insertion

Java Collections Framework

- **Interfaces et classes génériques**

- La notation `ArrayList<E>` permet de restreindre le type des instances ajoutées à l'`ArrayList`.
- `E` est un paramètre de type. Il est utilisé à l'intérieur de la classe, par exemple pour spécifier les arguments ou types de retour de méthodes.

```
public class ArrayList<E> ... {  
    public boolean add(E o) {...}  
    public E get(int i) {...}  
}
```



```
public class ArrayList<Carre> ... {  
    public boolean add(Carre o) {...}  
    public Carre get(int i) {...}  
}
```

- Le type `E` ne peut être un type primitif.
- Le compilateur vérifie que les instances passées comme paramètres aux méthodes d'`ArrayList` (p.ex. à la méthode `add`) sont compatibles avec `E`.
- On dit que la classe `ArrayList` est une **classe générique**. Le mécanisme *generics* sera discuté dans un chapitre ultérieur.

Table des Matières

- **Introduction**
- **Collections**
 - Listes
 - Ensembles
 - Files
- **Associations**

Listes

- **Tableau dynamique**

- L'exemple suivant illustre l'utilisation d'une liste, implémentée avec un tableau dynamique (ArrayList). Contrairement aux tableaux, la taille d'un tableau dynamique est variable.

```
List<String> etudiants = new ArrayList<>();  
etudiants.add("Véronique");  
etudiants.add("Jef");  
etudiants.add("Tom");  
etudiants.add("Luc");
```

```
for (String s: etudiants)  
    System.out.println(s);
```

affiche

Véronique
Jef
Tom
Luc

```
etudiants.remove("Luc");  
etudiants.add("Bruno");  
etudiants.add("Hadrien");  
etudiants.add("Souhaib");  
etudiants.add("Stéphane");  
etudiants.remove(5);  
etudiants.add(5, "Quentin");
```

```
System.out.println(etudiants.get(4));
```

affiche

Hadrien

```
System.out.println(etudiants.size());
```

affiche

7

supprime
sur base du
contenu

supprime
sur base de
l'index

Listes

- **Facilités pour la création de listes**

- `List.of(E...)` → génère liste immuable, pas de null

```
var l1 = List.of(1, 2, 3);  
l1.stream().forEach(System.out::println);  
l1.add(3); // UnsupportedOperationException  
l1.set(0, 4); // UnsupportedOperationException
```

- `Arrays.asList(T...)` → génère liste de taille fixe sur base d'un tableau (sur base classe interne `Arrays$ArrayList`)

```
var l2 = Arrays.asList(1, 2, 3);  
l2.stream().forEach(System.out::println);  
l2.add(3); // UnsupportedOperationException  
l2.set(0, 4);
```

- Attention avec les tableaux de primitifs

```
var l3 = List.of(new Integer[] {1, 2, 3});  
l3.stream().forEach(System.out::println)  
var l4 = List.of(new int[] {1, 2, 3});  
l4.stream().forEach(System.out::println)
```

-----> Liste d'un seul élément
(le tableau)

Listes

- **Ambiguïté possible avec `remove()`**
 - L'interface `List` contient deux méthodes `remove()`,
 - `remove(Object o)` [héritée de `Collection`] supprime un élément `e` tel que `Object.equals(o, e)`
 - `remove(int i)` supprime l'élément d'index `i`

```
List<Integer> l = new ArrayList();  
l.addAll(List.of(1, 2, 3));
```

```
l.remove(1);
```

→ Quel élément est retiré ?

- Pour forcer l'usage de `remove(Object)`, utiliser

```
l.remove(Integer.valueOf(1));
```


Listes

• Quand utiliser ArrayList ou LinkedList ?

- Indexation, `get(i)`
- Insertion au début, `add(0, e)`

```
List<String> l = Arrays.asList(new String[n]);
l = new (Array|Linked)List<String>(l);
long start = System.currentTimeMillis();
for (int i = 0; i < n; i++)
    l.get(i); ou l.add(0, i);
long duration = System.currentTimeMillis() - start;
```

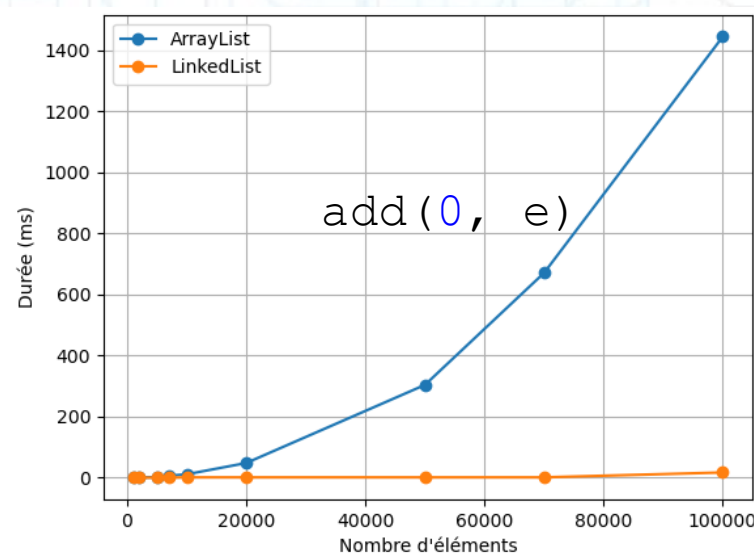
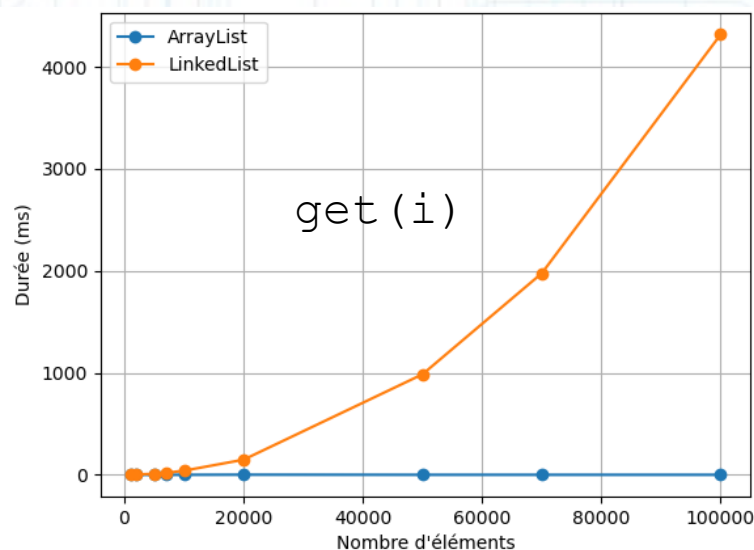


Table des Matières

- **Introduction**
- **Collections**
 - Listes
 - Ensembles
 - Files
- **Associations**

Ensembles

- **Unicité des éléments**

- Un ensemble ne peut contenir deux fois la même valeur, au sens d'`equals()`

```
Set<String> etudiants = new HashSet<>();  
etudiants.add("Guillaume");  
etudiants.add("Sébastien");  
etudiants.add("Valentin");  
etudiants.add("Guillaume"); /* ignoré, retourne false */  
  
System.out.println(etudiants.size()); /* 3 */
```

- Retourner les éléments différents d'une liste

```
List<String> etudiants = List.of(  
    "Guillaume", "Sébastien", "Valentin", "Guillaume"  
);  
Set<String> etudiantsUniques = new HashSet<>(etudiants);
```

Ensembles

- **Ordre des éléments**

- HashSet ne fournit aucune garantie sur cet ordre.
- TreeSet utilise l'ordre naturel (i.e. Comparable)
- LinkedHashMap conserve l'ordre d'insertion

```
Set<String> membres = new (Hash|Tree|LinkedHash) Set<>();  
membres.add("Valentin");  
membres.add("Guillaume");  
membres.add("Sébastien");  
membres.add("Jeremy");
```

```
for (String m: membres)  
    System.out.println(m);
```

HashSet

Valentin
Jeremy
Sébastien
Guillaume

TreeSet

Guillaume
Jeremy
Sébastien
Valentin

LinkedHashSet

Valentin
Guillaume
Sébastien
Jeremy

Table des Matières

- **Introduction**
- **Collections**
 - Listes
 - Ensembles
 - Files
- **Associations**

Pile (File LIFO)

- **Evaluateur d'expression RPN**

- La *Reverse Polish Notation* (RPN) est une notation permettant d'écrire des expressions arithmétiques sans parenthèses. Il s'agit d'une notation dite « *postfixe* ».
- Exemple : l'expression « $5 \times (3 + 4)$ » peut s'écrire « $3\ 4\ +\ 5\ \times$ » en notation RPN. Pour l'évaluer, on procède de la gauche à la droite. Dès qu'un opérateur binaire apparaît, on consomme les deux chiffres précédents et on y applique l'opérateur.
 - 3
 - 3 4
 - 3 4 + $\rightarrow$ 7
 - 7 5
 - 7 5 $\times$ $\rightarrow$ 35
- Comment écrire un évaluateur d'expression RPN ? En utilisant une pile...

Pile (File LIFO)

- **Evaluateur d'expression RPN**

- Voici une solution partielle utilisant une liste chaînée (LinkedList) ou un tableau dynamique (ArrayDeque) comme une pile, au travers de l'interface Deque.

```
public static double evaluate(String rpn) {  
    Deque<Double> stack = new LinkedList<>();  
    for (String token: rpn.split(" ")) {  
        switch (token) {  
            case "+": -> stack.push(stack.pop() + stack.pop());  
            case "*": -> stack.push(stack.pop() * stack.pop());  
            case "-": -> stack.push(-stack.pop() + stack.pop());  
            case "/": -> stack.push(1.0/stack.pop() * stack.pop());  
            default -> stack.push(Double.valueOf(token));  
        }  
    }  
    return stack.pop();  
}
```

- La solution ci-dessus ne traite pas les erreurs de syntaxe. Qu'ajouteriez-vous ?

Pile (File LIFO)

- Une pile avec une liste chaînée

```
public interface Action {  
    void perform(StringBuffer sb);  
    void unperform(StringBuffer sb);  
}
```

```
Deque<String> actions = new LinkedList<>();  
StringBuffer sb = new StringBuffer();
```

```
public void performAction(Action a) {  
    a.perform(sb);  
    actions.push(a);  
}
```

```
public void undo() {  
    actions.pop().unperform(sb);  
}
```

```
performAction(new Add("Bonjour"));  
performAction(new Add("World"));  
performAction(new Replace(0, 7, "Hello"));  
performAction(new Insert(7, " ", "));  
  
undo();  
undo();
```

Bonjour
BonjourWorld
HelloWorld
HelloWo, rld
HelloWorld
BonjourWorld

File de priorité

- **Exercice : ordonner des patients**
 - Supposons un hôpital recevant des patients à tout moment.
 - Ceux-ci peuvent être affectés de problèmes médicaux de **sévérités diverses** (basse, moyenne, élevée).
 - Lorsque le bloc opératoire est libéré d'un patient, il est nécessaire d'aller chercher le patient suivant, non pas en fonction de son ordre d'arrivée mais **selon la sévérité de son problème**.
 - On peut utiliser une `PriorityQueue` à cet effet. La sévérité d'un problème médical doit être modélisée à l'aide d'une énumération.

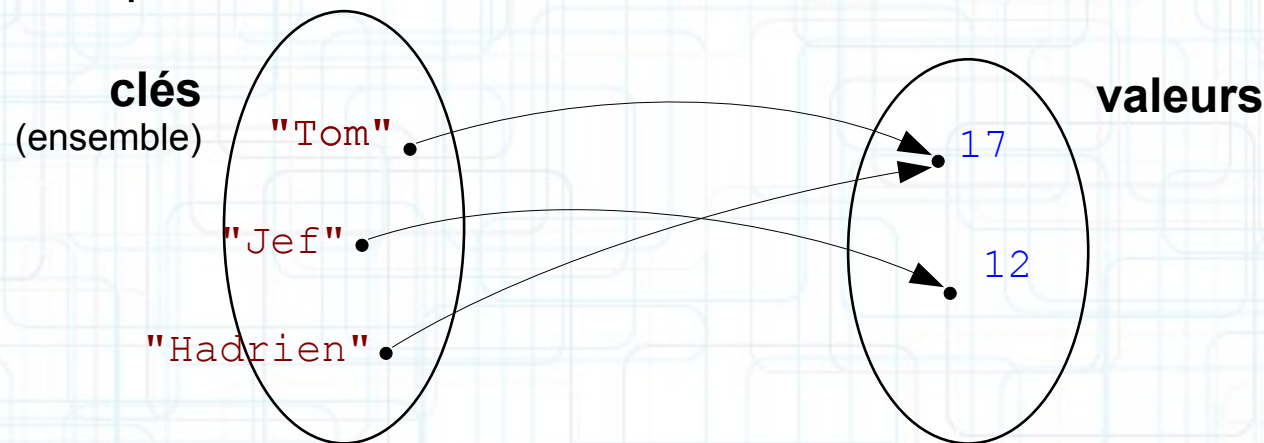
Table des Matières

- **Introduction**
- **Collections**
 - Listes
 - Ensembles
 - Files
- **Associations**

Associations

- **Tables de correspondance (Map)**

- Le JCF définit également des classes permettant de conserver les **associations** entre des **clés** (*keys*) et **valeurs** (*values*). On désigne parfois ces structures de données comme des tables de correspondance, des dictionnaires ou des tables de hachage⁽¹⁾.



- Il est possible de retrouver la valeur d'une association à l'aide de la clé qui lui est associée. Les clés ne peuvent être dupliquées (ensemble).
- Les clés et les valeurs des associations peuvent être récupérées indépendamment sous forme de collections.

⁽¹⁾ Table de hachage fait référence à une implémentation possible d'une table de correspondance.

Associations

- **Interface Map**

- L'interface **Map** (`java.util`) définit les opérations liées à la gestion d'une association.

```
public interface Map<K,V> {  
    void                clear();  
    boolean             containsKey(Object key);  
    boolean             containsValue(Object value);  
    Set<Map.Entry<K,V>>  
                        entrySet();  
    V                   get(Object key);  
    V                   getOrDefault(Object key, V defaultValue);  
    boolean             isEmpty();  
    Set<K>              keySet();  
    V                   put(K key, V value);  
    void               putAll(Map<? extends K, ? extends V> t);  
    V                   remove(Object key);  
    int                 size();  
    Collection<V>      values();  
    ...  
}
```


Associations

- **Table de hachage**

- L'exemple suivant illustre l'utilisation d'une table de correspondance implémentée avec une table de hachage (HashMap).

```
Map<String,Integer> resultats= new HashMap<>();
resultats.put("Bruno", 18);
resultats.put("Véronique", 16);
resultats.put("Jef", 12);
resultats.put("Hadrien", 17);
resultats.put("Tom", 17);

String key= "Tom";
if (resultats.containsKey(key)) {
    System.out.println("Résultat de \""+key+"\" : "+
        resultats.get(key));
}
```

clé

valeur

donne

Tom : 17

- Rappel : une clé est associée à une valeur unique !
 - utiliser 2 fois `put()` avec la même clé effectue un remplacement

Associations

- **Table de hachage**

- L'exemple suivant illustre la récupération des ensembles de clés et de valeurs d'une association avec les méthodes `keySet()` et `values()`.

```
Map<String,Integer> resultats= new HashMap<>();  
/* ... */
```

```
Set<String> keys= resultats.keySet();  
for (String k: keys)  
    System.out.println(k);
```



Bruno
Véronique
Jef
Hadrien
Tom

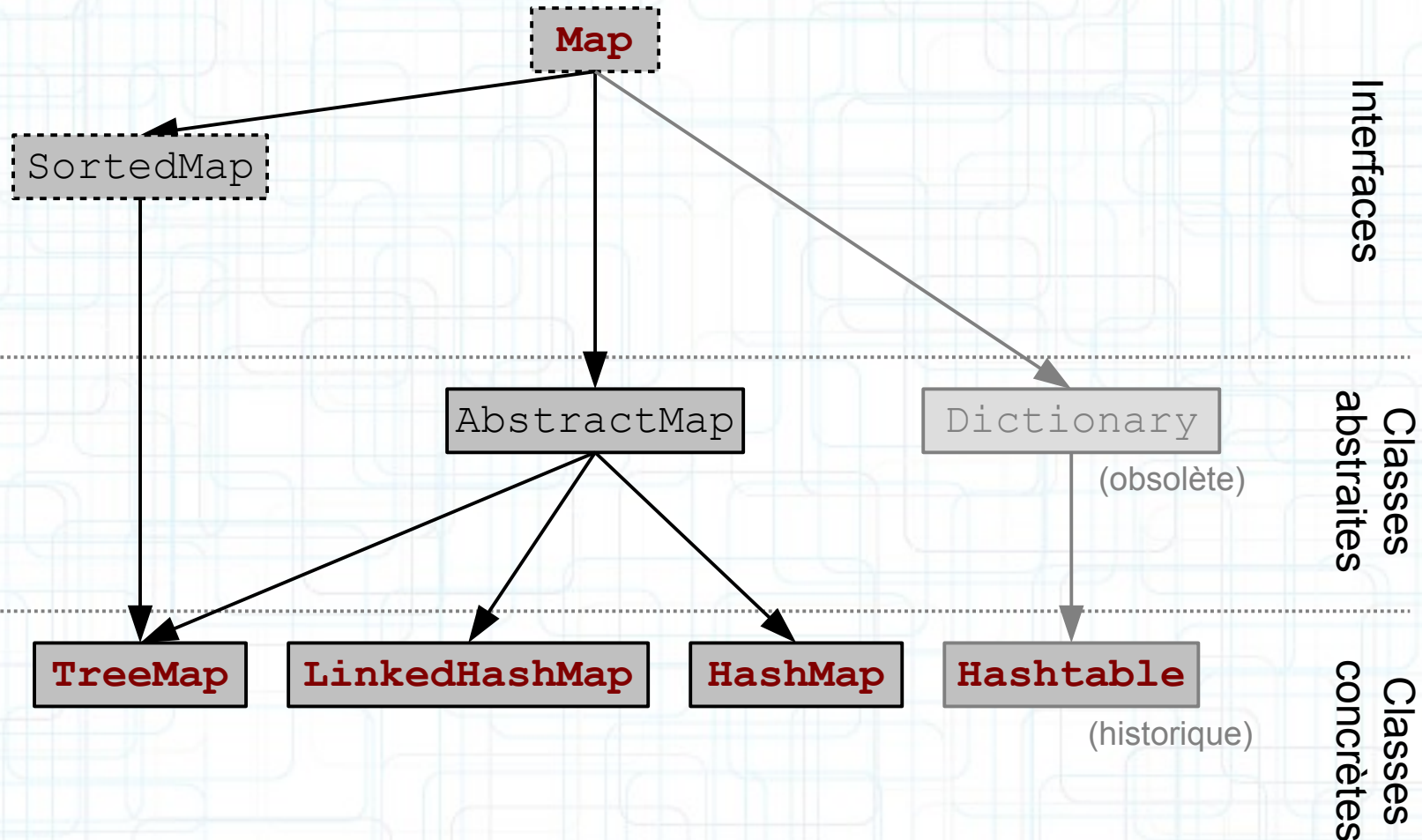
```
Collection<Integer> values= resultats.values();  
for (Integer v: values)  
    System.out.println(v);
```



18
15
12
17
17

Associations

- Hiérarchie de classes Map

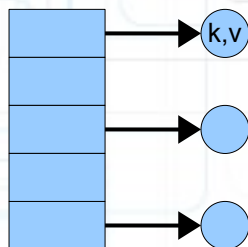


Associations

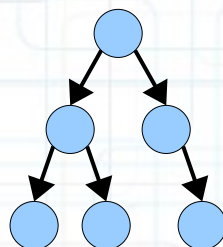
- Hiérarchie de classes Map

	clés		valeurs	synchr onisé	remarques
	null	ordon nées	null		
HashMap	✓		✓		table de hachage
TreeMap		✓ (1)	✓		arbre binaire (rouge-noir)
LinkedHashMap	✓	✓ (2)	✓		table de hachage + liste doublement chaînée
Hashtable				✓	table de hachage

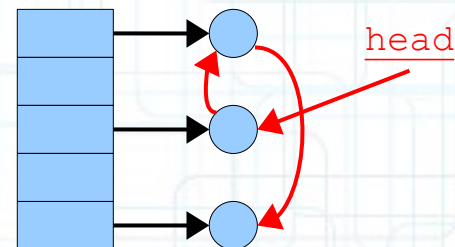
HashMap



TreeMap



LinkedHashMap



(1) ordre naturel ; (2) ordre d'insertion

Associations

- **Ordre des clés ?**

- HashMap ne fournit aucune garantie sur cet ordre.
- TreeMap utilise l'**ordre naturel** (i.e. Comparable)
- LinkedHashMap conserve l'**ordre d'insertion**

```
Map<String,Integer> resultats= new (Hash|Tree|LinkedHash)Map<>();  
resultats.put("Bruno", 18);  
resultats.put("Véronique", 16);  
resultats.put("Jef", 12);  
resultats.put("Hadrien", 17);  
resultats.put("Tom", 17);
```

```
Set<String> keys= resultats.keySet();  
for (String k: keys)  
    System.out.println(k);
```

HashMap

Bruno
Tom
Hadrien
Véronique
Jef

TreeMap

Bruno
Hadrien
Jef
Tom
Véronique

LinkedHashMap

Bruno
Véronique
Jef
Hadrien
Tom

Associations

- Fréquence des mots dans un texte

```
Map<String,Integer> frequence = new HashMap<>();

Path filePath = Path.of("alice.txt");
try (BufferedReader reader =
    Files.newBufferedReader(filePath)) {
    String line;
    while ((line = reader.readLine()) != null) {
        String[] words = line.split("\\W+");

        for (String word: words) {
            if (!word.isEmpty()) {
                word = word.toLowerCase();
                int oldCount = wordCounts.getOrDefault(word, 0);
                wordCounts.put(word, oldCount + 1);
            }
        }
    }

    var entryList = new ArrayList<>(wordCounts.entrySet());
    entryList.sort((e1, e2) -> e2.getValue() - e1.getValue());
    for (Map.Entry e: entryList)
        System.out.println(e.getKey() + "\t" + e.getValue());
}
```

the	1839
and	940
to	811
a	695
of	637
it	607
she	549
i	524
you	471
said	460
in	433
alice	402
(...)	